



Programming Foundations Level 1



Programming Foundations

Level 1

Programming Languages

Part 1



Programming Foundations

Level 1

Programming Languages and Levels

Programming Languages & Levels: From Machine Code to High-Level

A **programming language** is a tool we use to write instructions (**code**) so the computer performs a task.

Those instructions can exist at different **levels** of abstraction — from raw **binary (machine language)** up to **human-readable high-level languages** — and we need **translator software** (compilers/interpreters) to bridge from what we write to what the CPU actually runs.

We have hundreds of Programming languages





Programming Foundations

Level 1

What is Programming Language?

Programming Language Is only a **TOOL** to help you write instructions to computer to perform certain operation/Task.

Instructions are called **code**

A tool to express instructions that a computer will execute.

The instructions we write are called **code**.

Write a program to print



Machine Language (Binary/Hex): direct CPU instructions, e.g., a stream of 0/1 representing "Hello, World!" — the computer can run this directly, but it's unreadable for humans.

Machine Language:

Computer programming language consisting of **binary or hexadecimal instructions** which a computer can respond to **directly**.



Binary

```
01001000 01100101 01101100
01101100 01101111 00101100
00100000 01010111 01101111
01110010 01101100 01100100
00100001
```

Machine Language:

```
0x55 0x89 0xe5 0xe8 0xfc 0xff 0xff 0xff
0x83 0xf8 0x41 0x75 0x68 0x0d 0x00 0x00
0x00 0x00 0xe8 0xfc 0xff 0xff 0xff 0x83
0xb8 0x00 0x04 0xc4 0x00 0x00 0x00 0x89
0xec 0x5d 0xc3
```

Assembly Language: symbolic mnemonics (e.g., *MOV*, *CALL*) that map almost 1-to-1 to machine instructions; easier than binary, but still hardware-specific.

Assembly Language:

```
push %ebp
mov %esp,%ebp
call 0x8048298
cmp $0x41,%eax
jne 0x80483ce
push $0x80484b0
call 0x80482c8
add $0x4,%esp
mov $0x0,%eax
mov %ebp,%esp
pop %ebp
ret
```



Machine Code vs Assembly Language:

Machine Language

```
0x55 0x89 0xe5 0xe8 0xfc 0xff 0xff 0xff  
0x83 0xf8 0x41 0x75 0x68 0x0d 0x00 0x00  
0x00 0x00 0xe8 0xfc 0xff 0xff 0xff 0x83  
0xb8 0x00 0x04 0xc4 0x00 0x00 0x00 0x89  
0xec 0x5d 0xc3
```

Assembly Language

```
push %ebp  
mov %esp,%ebp  
call 0x8048298  
cmp $0x41,%eax  
jne 0x80483ce  
push $0x80484b0  
call 0x80482c8  
add $0x4,%esp  
mov $0x0,%eax  
mov %ebp,%esp  
pop %ebp  
ret
```



High-Level Languages: human-readable, portable syntax (e.g., PRINT "Hello, World!" in BASIC, or Console.WriteLine("Hello, World!");).

High level Languages



"Basic" Language

PRINT "Hello, World!"



The following contrast all three for "Hello, World!" and show **Machine vs Assembly vs High level Languages**, highlighting the jump in readability.

Machine vs Assembly vs Basic:

Low Level Languages



Machine Code

```
0x55 0x89 0xe5 0xe8 0xfc 0xff 0xff 0xff
0x83 0xf8 0x41 0x75 0x68 0x0d 0x00 0x00
0x00 0x00 0xe8 0xfc 0xff 0xff 0xff 0x83
0xb8 0x00 0x04 0xc4 0x00 0x00 0x00 0x89
0xec 0x5d 0xc3
```



Assembly Language

```
push %ebp
mov %esp,%ebp
call 0x8048298
cmp $0x41,%eax
jne 0x80483ce
push $0x80484b0
call 0x80482c8
add $0x4,%esp
mov $0x0,%eax
mov %ebp,%esp
pop %ebp
ret
```

High Level Language

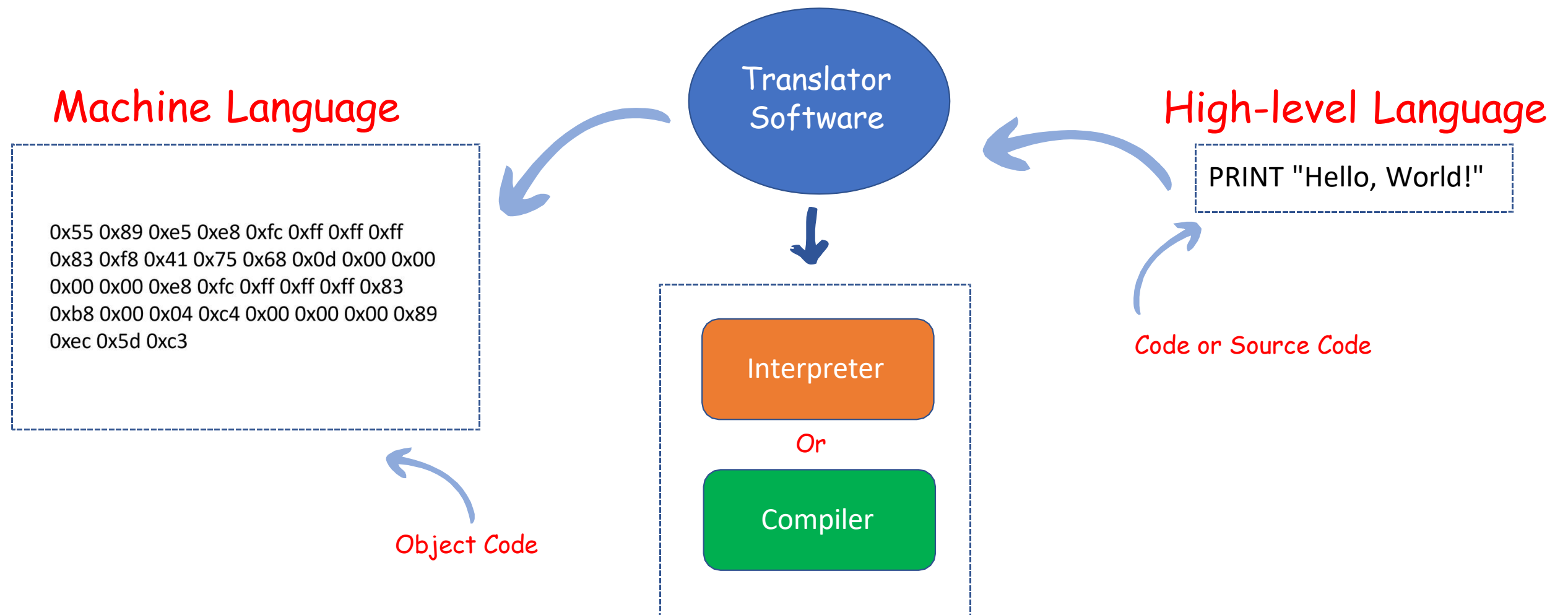


"Basic" Language

```
PRINT "Hello, World!"
```

Machine vs Assembly vs Basic:

You need a Translator!



Machine vs Assembly vs Basic:

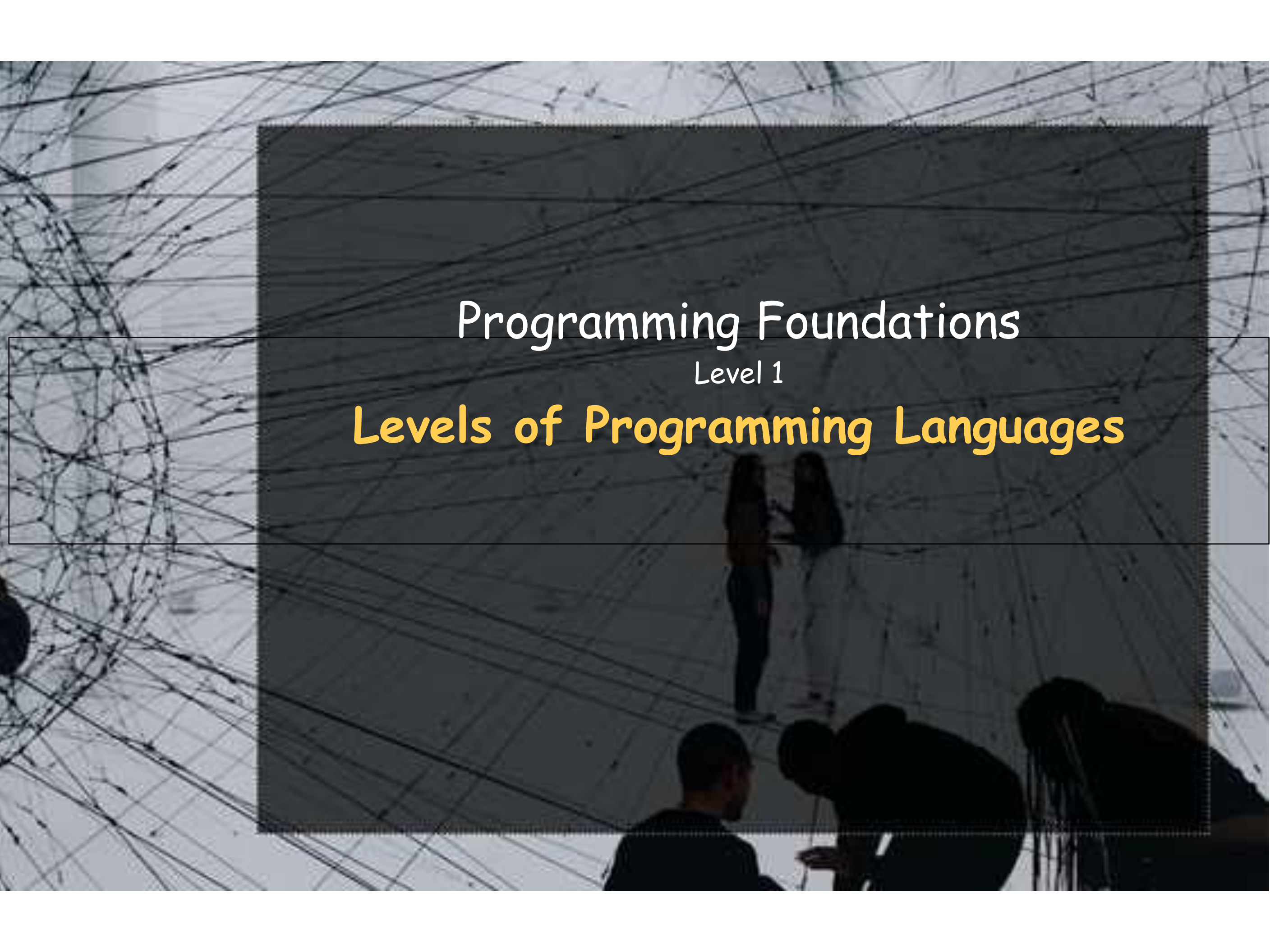
Translator Software: either a **Compiler** or an **Interpreter** (the slides show both paths).

Compiler: translates the whole program to object/machine code **before** execution.

Interpreter: reads and executes **line by line** at runtime.

Source Code: what you write in a high-level or assembly language.

Object Code: the **translated/compiled** low-level form (machine code) that the computer can execute.

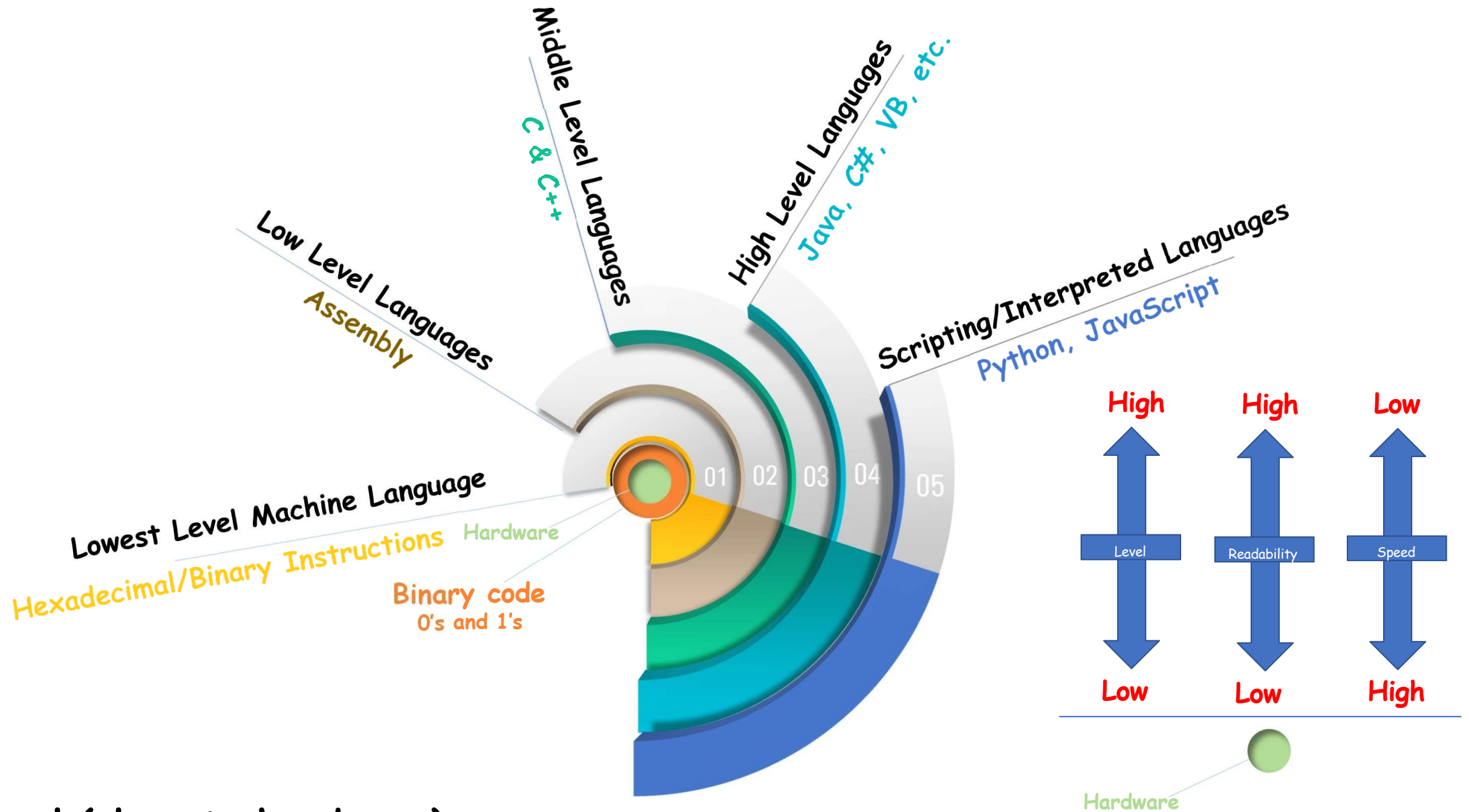


Programming Foundations

Level 1

Levels of Programming Languages

Machine vs Assembly vs Basic:



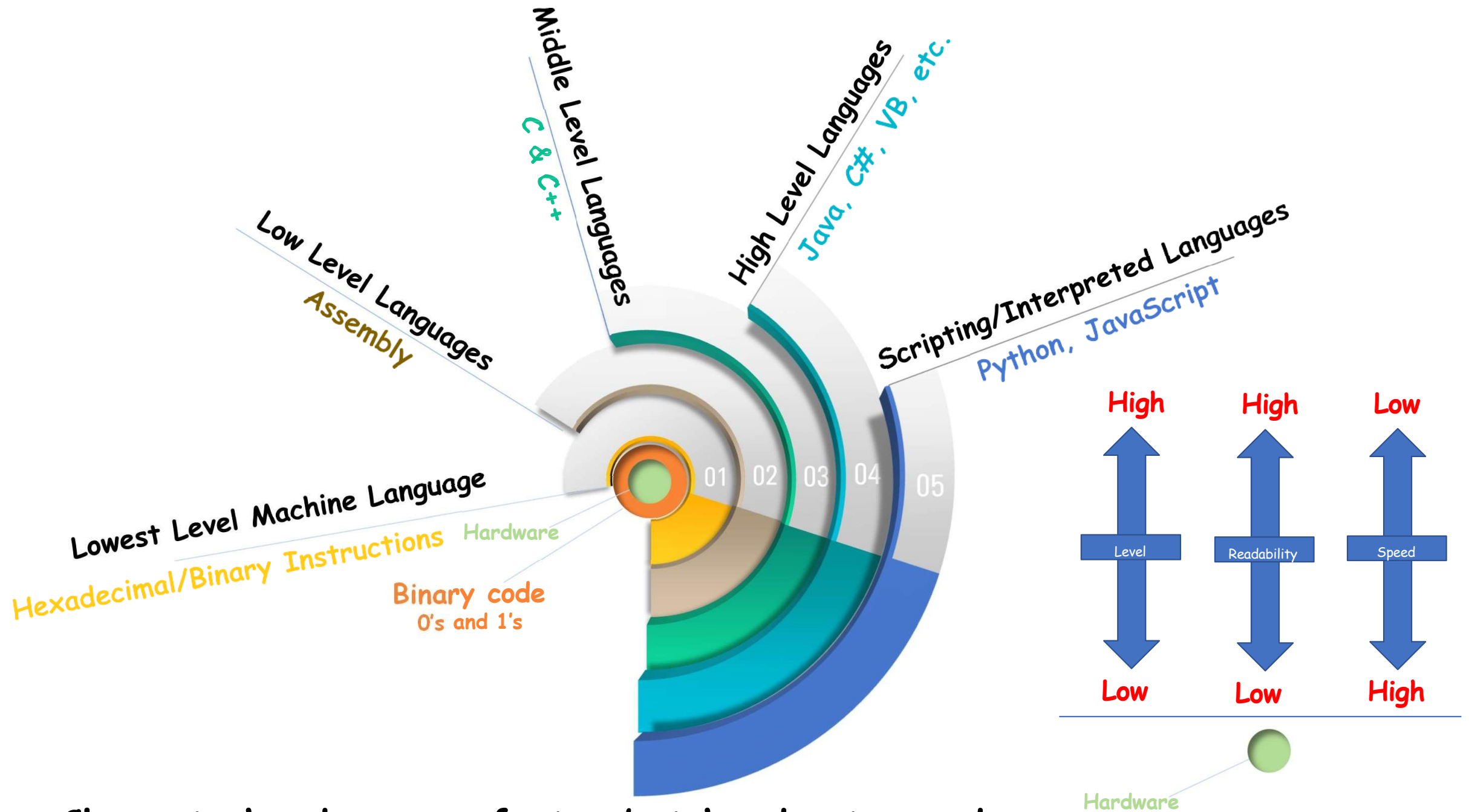
Low Level (close to hardware):

Binary/Assembly → High speed, Low readability.

High Level (farther from hardware):

BASIC/C/Java/C#... → High readability, Lower raw speed (more abstraction/overhead).

Machine vs Assembly vs Basic:



Key idea: **Closer to hardware** → **faster but harder to read**.

Farther → **easier to read/write/maintain** but potentially slower. Real-world speed also depends on compiler quality, runtime, JIT, optimization flags, and the problem itself.

When is a Language “Fast” or “Slow”?

- ◆ **Fast (typically):**
 - Closer to machine code (assembly/C in many cases).
 - Compiled ahead-of-time with strong optimizations; minimal runtime overhead.
- ◆ **Slow (relatively):**
 - Heavier runtime interpretation/JIT, frequent dynamic checks, or layers of abstraction.
- ◆ **But:** highly optimized compilers/JITs can make some high-level languages **very fast** for many workloads. Choose based on **use case + ecosystem**.

Interconnection

- ◆ **Machine** → **Assembly** → **High-Level** forms a spectrum: **speed** ↑ as we go down to hardware, **readability** ↑ as we go up.
- ◆ **Source Code** is what we write; **Object Code** is what runs.
A **compiler/interpreter** connects them.
- ◆ “Hello, World!” example across levels shows **why translators are essential** and **how abstraction helps humans**.

By understanding these links, learners can pick the **right language level** and toolchain for performance, portability, and productivity. 🚀

Activity

Instructions:

Write your answers clearly and concisely in your own words.

- 1 What is a **Programming Language**?
- 2 What is **Code**?
- 3 What is **Source Code**?
- 4 What is **Object Code**?
- 5 Why do we need **Compilers or Interpreters**?
- 6 When is a language considered **Fast**?
- 7 When is a language considered **Slow**?
- 8 What is a **High-Level Language**?
- 9 What is a **Low-Level Language**?
- 10 When do we consider a language **human-readable**?



Activity Solution



Solution:

1 Programming Language:

A tool used by programmers to write instructions that tell a computer what to do.

2 Code:

The set of written instructions or commands created using a programming language.

3 Source Code:

The human-readable form of a program that the programmer writes before it's translated.

4 Object Code:

The machine-readable form produced after the source code is translated (compiled).

5 Why Compilers or Interpreters:

Because computers only understand **machine code (0s and 1s)**, we need translators to convert human-readable code into a format the CPU can execute.

6 When Fast:

A language is considered **fast** when it runs close to the hardware (low-level) or when its compiler produces highly optimized machine code.

7 When Slow:

A language is **slow** when it has more abstraction, is interpreted line by line, or depends on extra runtime layers that add overhead.

8 High-Level Language:

A programming language that is **easy to read and write**, close to human language, and hides most hardware details (e.g., Python, C#, Java).

9 Low-Level Language:

A programming language that is **close to machine code**, gives full control over hardware, but is harder to read (e.g., Assembly, Machine Language).

10 Human-Readable Language:

A language is human-readable when its syntax uses **words, symbols, and structure** that are easy for people to understand — not binary or numeric codes.

Conclusion

- ✓ A programming language is a tool; the instructions we write are **code**.
- ✓ Source code must be translated into **object/machine code** via a **compiler** or **interpreter** to run.
- ✓ There's a continuum from **machine** → **assembly** → **high-level** where we trade **raw speed** for **human readability** and **productivity**.
- ✓ Your choice of level depends on **performance needs**, **portability**, **team skills**, and **tooling**.

Thank you 😊